

Open in app ↗

Sign up

Sign in

Medium

 Search

High Performance Memory Management: Arena Allocators

12 min read · Jul 30, 2025



Ng Song Guan

Follow



Listen



Share

In performance-critical applications, even small inefficiencies in memory management can be intolerable. Unlike languages such as Java and Python, which rely on garbage collection, C++ gives you precise control over memory allocation and deallocation, making it a popular choice in these domains.

In this article, I'll be discussing memory management using arena allocators. I'll show you how they work, how to write one in C++, and how they can boost performance. But first, let's take a quick look at how memory allocation typically works in C++.

C++ Memory Management 101

When you create objects in C++, you generally have two choices based on their storage duration:

- **Automatic Storage (Stack Allocation):** Objects are allocated on the stack, and their lifetimes are tied to their scope.
- **Dynamic Storage (Heap Allocation):** Objects are allocated on the free store (heap), and they survive beyond their creation scope until explicitly deallocated.

Consider a simple struct `Foo`.

```
struct Foo {  
    int x;  
    int y;  
};
```

Here's how you can allocate it:

```
void func() {  
    /* Stack allocation: cheap and automatic. */  
    // f1 is destroyed automatically at the end of this scope.  
    Foo f1{3, 4};  
  
    /* Heap allocation: flexible but more overhead. */  
    // f2 persists after the function returns, but requires manual deletion.  
    Foo* f2 = new Foo{3, 4}; // 'old' style  
  
    // Modern C++ with RAII: automatic cleanup via smart pointer.  
    std::unique_ptr<Foo> f3 = std::make_unique<Foo>(3, 4);  
    // f3 will automatically free the heap memory when it goes out of scope.  
  
    // STL Containers may manage dynamic allocations internally.  
    // Example: std::vector stores objects on the heap.  
    std::vector<Foo> vec;  
    vec.push_back(Foo{3, 4}); // Allocates Foo inside vector's heap buffer  
    vec.push_back(Foo{5, 6}); // Vector resizes heap buffer as needed.  
    // vec and its contents are cleaned up  
    // automatically when going out of scope.  
}
```

Stack allocation is very efficient because it simply adjusts the stack pointer register (e.g., decrementing `rsp` on x86). In contrast, dynamic heap allocation has more overhead. The `new` keyword invokes the system's allocator, typically `malloc()` on Linux.

What does `malloc()` actually do?

`malloc()` is a library function provided by glibc on Linux¹ (not a system call). It serves as a general-purpose memory allocator, designed to handle diverse allocation patterns across threads and programs. Its implementation is complex, spanning thousands of lines of code. At a high level, a typical call to `malloc()` involves the following steps:

1. Search the free list for an existing block that fits the requested size.
2. If none is found, attempt to carve a new block from the available heap space.
3. If the heap is too small, call the `brk()` system call to extend the program's heap.
4. For large allocations, bypass the heap and use `mmap()` to request memory directly from the OS.

Although `malloc()` has been written in an optimized manner, its general-purpose nature makes it slower and unpredictable as compared to stack allocation. The time it takes can vary depending on the heap state. It might involve scanning free lists or even triggering system calls, which add latency due to context switching. Such variability is unacceptable in many high-performance workloads².

Additionally, every call to `malloc()` should eventually be paired with a corresponding `free()` call. But `free()` also introduces overhead: it must update bookkeeping structures and return the block to the internal free list.

Another issue is memory fragmentation. `malloc()` can often scatter allocations across the address space. In workloads which involves allocating many small objects that are accessed in rapid succession, this hurts cache locality as the objects have poor spatial proximity.

To improve performance, we want to minimize the number of individual `malloc()` calls. This is where arena allocation comes in.

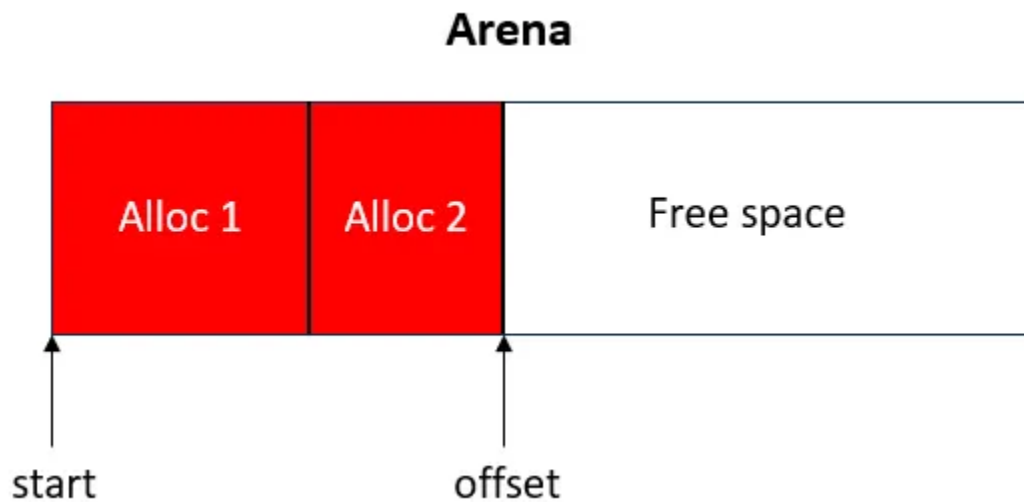
What is an Arena?

An arena is actually very simple in concept — we pre-allocate (typically using a single `malloc()`) a big block of memory upfront, and then perform subsequent

allocations from this block.

The type of arena I'll be discussing is also known as a linear allocator or bump allocator. It works by maintaining a current offset within the pre-allocated block, which initially starts at 0. Every time an object is allocated, the arena:

1. Returns a pointer to the current offset
2. Increments ('bumps') this offset by the size of the allocated object



Individual deallocations are not supported in arenas. Instead, the entire block of memory is freed at once, similar to tearing down the entire stack frame at the end of a function call.

Let's see how we can implement an arena in C++.

```
#pragma once
#include <cstddef>
#include <new>
#include <memory>

class Arena {
public:
    // Constructor: allocate a raw memory block of given size
    explicit Arena(std::size_t size)
```

```
        : buffer_(static_cast<char*>(::operator new(size))),
          capacity_(size), offset_(0) {}

// Destructor: free the memory block
~Arena() {
    ::operator delete(buffer_);
}

// Allocate memory of `size` bytes with specified `alignment`
void* allocate(std::size_t size, std::size_t alignment) {
    char* current_ptr = buffer_ + offset_;
    std::size_t space = capacity_ - offset_;
    void* aligned_ptr = current_ptr;

    // Align the ptr to the specified alignment
    if (std::align(alignment, size, aligned_ptr, space) == nullptr) {
        throw std::bad_alloc();
    }

    // Bump the offset of the arena
    offset_ = static_cast<char*>(aligned_ptr) - buffer_ + size;

    return aligned_ptr;
}

// Reset arena to reuse the memory block from the start
void reset() {
    offset_ = 0;
}

// Non-copyable, non-movable type
Arena(const Arena&) = delete;
Arena(Arena&&) = delete;

private:
    char* buffer_;           // Start of the allocated memory block
    std::size_t capacity_;   // Total size of the block
    std::size_t offset_;     // Current allocation offset
};
```

Characteristics of this arena:

- There's no `deallocate()` function. We cannot deallocate individually.

- The `reset()` function sets the internal offset back to 0, invalidating all previous allocations and allowing the arena buffer to be reused from the start.
- The entire block of memory is automatically freed when the `Arena` object goes out of scope.
- During allocation, we use `std::align` to ensure proper alignment.

Example usage:

```
void parent_func() {
    Arena arena(1024); // Create arena with 1024 bytes
    func(arena);
    // Memory is automatically freed when `arena` goes out of scope
}

void func(Arena& arena) {
    // This replaces use of malloc
    // auto* ptr = malloc(sizeof(Foo));
    auto* ptr = arena.allocate(sizeof(Foo), alignof(Foo));
    auto* foo_ptr = new (ptr) Foo({3, 4});
}
```

In this example, we allocate raw memory from the arena, and then use `placement new` to construct `Foo` in-place. It's important to remember that object creation in C++ is a two-step process:

1. Allocate raw memory.
2. Construct the object.

Normally, this is abstracted away by the `new` keyword or `std::make_unique`, but here we handle them separately.

From this implementation, the advantages of arenas should also be quite clear:

- Each allocation is fast and deterministic as it is just a simple pointer bump with no additional bookkeeping or syscall overhead (similar to stack allocation).

- Potentially better cache locality due to allocating from a contiguous memory block.
- Cheap deallocation by freeing the entire block at once.

The tradeoff is flexibility. You can't free individual objects, so all allocations should have similar lifetimes.

Get Ng Song Guan's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐

Remember me for faster sign in

Additional notes on `Arena` class

- It uses a minimalist C-style design. `Arena` only handles raw memory allocation. Users must handle the construction and destruction of objects. This makes it best for trivially destructible types (e.g., PODs) where no explicit cleanup is required. Later, I'll show how we can write a more object-oriented C++ arena.
- It is not thread-safe. This implementation can only be used in single-threaded contexts unless you add some form of synchronization mechanism.
- `Arena` uses `operator new` for allocating initial block of memory. For low-latency systems, you can use `mmap()` with `MAP_POPULATE` instead to ensure memory is backed by physical pages upfront. This reduces runtime page faults at the cost of higher initial memory commitment, trading space for predictability.

Integrating Arena with STL container (custom allocator)

By default, STL containers use `std::allocator`, which relies on `operator new` (and ultimately `malloc()`) for each dynamic allocation. We can write a custom allocator

that uses `Arena` to allocate instead and use this in STL containers.

Here's a minimal example of a custom stateful allocator that delegates to our `Arena` class:

```
#pragma once
#include "arena.hpp"

// Implement requirements of a custom allocator class
// https://en.cppreference.com/w/cpp/named_req/Allocator.html
template <typename T>
class ArenaAllocator {

public:
    using value_type = T;

    explicit ArenaAllocator(Arena* arena) noexcept : arena_(arena) {}

    template <typename U>
    ArenaAllocator(const ArenaAllocator<U>& other) noexcept : arena_(other.arena_) {}

    template <typename U>
    friend class ArenaAllocator;

    T* allocate(std::size_t n) {
        return static_cast<T*>(arena_>allocate(n * sizeof(T), alignof(T)));
    }

    void deallocate(T* /*p*/, std::size_t /*n*/) noexcept {
        // No-op; arena manages memory lifetime
    }

    bool operator==(const ArenaAllocator& other) const noexcept {
        return arena_ == other.arena_;
    }

    bool operator!=(const ArenaAllocator& other) const noexcept {
        return !(*this == other);
    }

private:
    Arena* arena_;
};
```


We can now use this allocator with an STL container, such as `std::unordered_map`:

```
int main() {
    Arena arena(1024);
    ArenaAllocator<std::pair<const int,int>> my_alloc(&arena);

    // std::unordered_map using our ArenaAllocator
    std::unordered_map<int,int,
        std::hash<int>,
        std::equal_to<int>,
        ArenaAllocator<std::pair<const int,int>>
    > umap(0, std::hash<int>(), std::equal_to<int>(), my_alloc);

    for (int i = 0; i < 12; i++) {
        umap[i] = i;
    }
}
```

`std::unordered_map` is implemented as a separate-chaining hash table. On insertion of a new element, it always needs to dynamically allocate a new node via `operator new` when using the default `std::allocator`. With our custom `ArenaAllocator`, we bypass dynamic allocations, and just allocate these new nodes directly from our arena. This improves performance.

To prove this, I wrote a simple micro-benchmark using google benchmark, benchmarking time taken to insert elements into `std::unordered_map`. The benchmark code can be found here: <https://github.com/sgn00/sgarena/blob/main/bench.cpp>

Benchmark	Time	CPU	Iterations
BM_UnorderedMap_Insert/100	4661 ns	4653 ns	152424
BM_UnorderedMap_Insert/1000	42135 ns	42062 ns	16704
BM_UnorderedMap_Insert/10000	448031 ns	447597 ns	1579
BM_ArenaUnorderedMap_Insert/100	2823 ns	2821 ns	248437

BM_ArenaUnorderedMap_Insert/1000	25477 ns	25445 ns	27417
BM_ArenaUnorderedMap_Insert/10000	274952 ns	274785 ns	2587

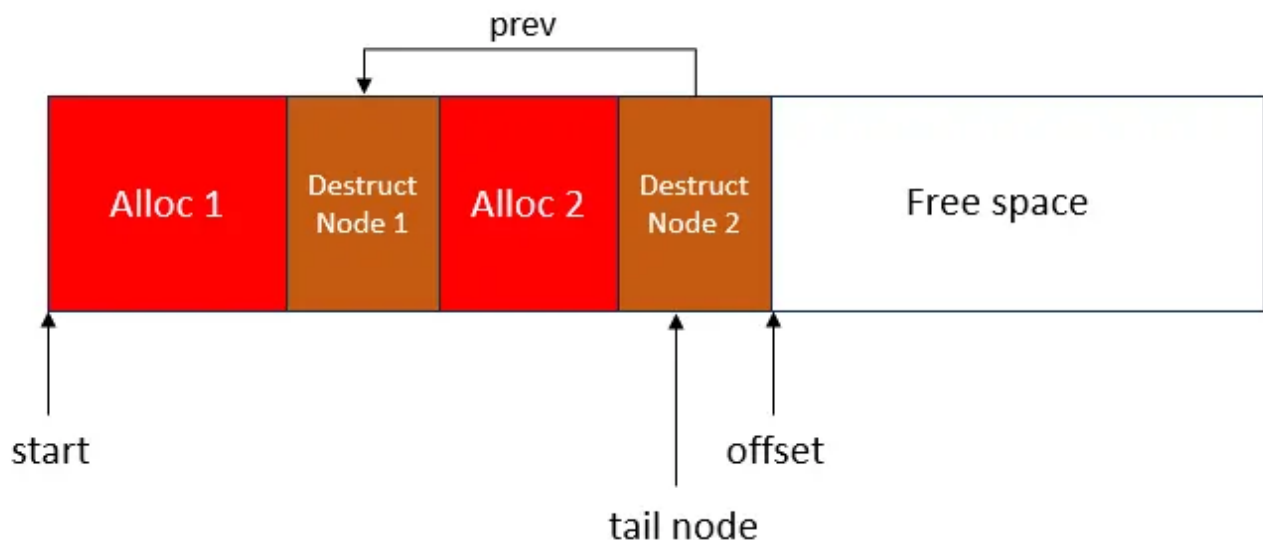
The arena-backed version consistently outperforms the version using the default allocator. This is an easy enhancement to `std::unordered_map` that can be useful in applications where performance matters. That said, in high performance scenarios, it may be worth considering an alternative hash table implementation altogether as `std::unordered_map` is known to be convenient, but not the fastest.

A More User Friendly C++ Arena

As I mentioned, our current `Arena` class is rather low level and C-style as it only handles raw memory allocation, leaving object construction and destruction entirely to the caller. This is fine for trivially destructible types, but we can do better in C++.

To support object construction is simple, we just need to move the `placement new` call into the arena's API. The challenge is destruction — we need to call the destructors of all constructed objects automatically when the arena is destroyed. To solve this, we will need to save the destructors in a list. But where do we get the space for this?

From the arena itself! We can create a new node type which saves the destructor to be called, and these nodes are also allocated in the arena.



Arena with extra linked list of destructor nodes embedded

As shown in the diagram, the destruct nodes form a reversed linked list. When the arena is destroyed, the arena's destructor will walk this list to call destructors opposite to the order of construction.

Here is our new arena implementation:

```
#pragma once
#include <stddef>
#include <memory>
#include <new>
#include <type_traits>

class ObjArena {

public:
    explicit ObjArena(std::size_t size)
        : buffer_(static_cast<char*>(::operator new(size))),
          capacity_(size), offset_(0) {}

    ~ObjArena() {
        call_destructors();
        ::operator delete(buffer_);
    }

    template <typename T, typename... Args>
    T* create(Args&&... args) {
        // Allocate space for obj
        auto [ptr, new_offset] = allocate(offset_, sizeof(T), alignof(T));

        if constexpr (std::is_trivially_destructible_v<T>) {
            // If T has trivial destructor, then just construct object
            // as per normal
            T* obj = new (ptr) T(std::forward<Args>(args)...);
            offset_ = new_offset;
            return obj;
        } else {
            // Else we need to create a DestructNode,
            // to store destructor to be called later

            // Allocate space for DestructNode
            auto [dnode_ptr, final_offset] = allocate(new_offset,
                sizeof(DestructNode), alignof(DestructNode));
```

```
// Now construct T only after successful allocation
T* obj = new (ptr) T(std::forward<Args>(args)...);

auto dtor_call = [](void* p) {
    static_cast<T*>(p)->~T();
};
DestructNode* new_node = new (dnode_ptr) DestructNode{dtor_call,
                                                    tail_, obj};

tail_ = new_node;

// Commit offset after successful allocation and construction
offset_ = final_offset;
return obj;
}
}

void reset() {
    call_destructors();
    offset_ = 0;
}

// Non-copyable, non-movable type
ObjArena(const ObjArena&) = delete;
ObjArena(ObjArena&&) = delete;

private:
    struct DestructNode {
        void (*dtor)(void*);
        DestructNode* prev;
        void* obj;
    };

    char* buffer_;
    std::size_t capacity_;
    std::size_t offset_;
    DestructNode* tail_ = nullptr;

    void call_destructors() {
        while (tail_) {
            tail_->dtor(tail_->obj);
            tail_ = tail_->prev;
        }
    }

    // Check if we have memory of `size` bytes with specified `alignment`
    std::pair<void*, std::size_t> allocate(std::size_t curr_offset,
                                           std::size_t size, std::size_t alignment) {
```

```
    char* current_ptr = buffer_ + curr_offset;
    std::size_t space = capacity_ - curr_offset;
    void* aligned_ptr = current_ptr;

    // Align the ptr to the specified alignment
    if (std::align(alignment, size, aligned_ptr, space) == nullptr) {
        throw std::bad_alloc();
    }

    auto new_offset = static_cast<char*>(aligned_ptr) - buffer_ + size;

    return {aligned_ptr, new_offset};
}
};
```

Example Usage:

```
struct Bar {
    Bar();
    ~Bar() {
        std::cout << "non-trivial dtor" << std::endl;
    }
};

void func() {
    ObjArena arena(1024);
    Bar* bar = arena.create<Bar>();
    std::cout << "End of scope" << std::endl;
    /*
    Prints:
    End of scope
    non-trivial dtor
    */
}
```

Our `ObjArena` supports any type `T`, creating destructor nodes only for those with non-trivial destructors. However, there are tradeoffs to supporting non-trivial destructors. Resetting or destroying the arena is now $O(n)$ in time complexity, as it calls every destructor individually.

Note that there are some changes in `ObjArena`'s `allocate()` compared to `Arena` as we want `ObjArena`'s `create()` function to be strong exception-safe. If allocation or construction throws exceptions, the arena's offset isn't updated, preventing inconsistent state.

Running out of memory in an arena

Our current arena allocates a fixed block of memory upfront during initialization, so it has limited space. If we try to allocate beyond this limit, the arena throws an error. This is in contrast with `malloc()`, which won't fail in most cases on a Linux 64 bit platform. There are several ways to handle running out of space in the arena:

1. **Fail** — Either throw an exception or return a `nullptr` (our current way). It's not a bad option. If you know your workload's memory requirements in advance, there's no need to overcomplicate things. Let the caller treat this as an exceptional scenario. Also, due to the virtual memory system and demand paging, on most 64 bit platforms, the OS allows us to allocate a very large virtual memory block upfront without issue.
2. **Grow the arena** — Instead of a single block of memory, the arena can manage multiple blocks, allocating new ones as needed and chaining them like a linked list. This is similar to the internal implementation of `std::deque`. It adds complexity but removes the fixed size limitation.
3. **Fallback to `malloc()`** — If the arena runs out of space, it can internally fall back to `malloc()`. This means the arena must keep track of which allocations came from the arena and which came from `malloc()`, and free them accordingly, complicating management.

In general, I prefer to keep arenas as simple as possible. Over-engineering to handle every edge case risks reinventing (a bad version of) `malloc()`.

Real-World Use of Arenas

Arenas are frequently used in performance-critical contexts where many objects share the same lifetime and can be freed together.

For example in game development, each iteration of the game loop renders a frame, and objects allocated during the frame can all be discarded at the end. An arena is a perfect fit here as we can allocate in the arena during the frame, and then reset the arena at the end of the frame.

Another common use case is parsing workloads, such as in compilers or Google Protobuf. These systems create many small, short-lived objects and use arenas heavily to avoid the cost of per-object heap allocation. In fact, my arena implementation is based off of [Google Protobuf](#).

In low-latency trading systems, heap allocations during runtime are often avoided altogether. As far as possible, all required memory is pre-allocated during initialization. That memory is then never freed until program shutdown. This is essentially a permanent arena that lives for the duration of the application.

`std::vector` **as an arena?**

You've probably already used `std::vector` in a way that is conceptually similar to an arena.

Consider the following:

```
void process_frame() {
    std::vector<Foo> arena_vec;
    // Preallocates memory for 1000 Foo
    // (but don't construct them)
    arena_vec.reserve(1000);
    process(arena_vec);
    // Memory and objects freed automatically on scope exit
}

void process(std::vector<Foo>& vec) {
    ...
    auto* ptr = create_obj(vec);
    ...
}

Foo* create_obj(std::vector<Foo>& vec) {
    ...
}
```

```
// Don't allow allocating beyond capacity of the vector
// or vector will resize and invalidate pointers!
if (vec.size() == vec.capacity()) {
    throw std::runtime_error("Out of space");
}

vec.emplace_back({5, 6});
return &vec.back();
}
```

This isn't strictly speaking an arena, but it shares the same philosophy: pre-allocate memory upfront, subsequent allocations come from this block, free everything in one go at the end.

When working with a single homogeneous type, containers like `std::vector` with `reserve()` are often all you need to reduce allocation overhead and you don't actually need to roll your own arena.

To sum up, in high-performance, low-latency workloads, the general rule is to avoid dynamic allocations altogether. But when dynamic allocations are unavoidable, custom allocators are used to meet the strict performance and latency requirements. Arenas are just one kind of custom allocation technique. Others include stack allocators, pool allocators, and even hybrid schemes like arena-pool allocators. Each comes with trade-offs and is suited to different use cases.

The full code for my arena implementations can be found here: <https://github.com/sgn00/sgarena>

[1] There are alternatives to glibc malloc that can be used as the system allocator, such as jemalloc and tcmalloc, each offering different characteristics. For example, jemalloc doesn't use `sbrk/brk` and only uses `mmap` internally.

[2] `malloc()` can take anywhere from tens of ns up to a few us. These latency spikes are undesirable in low-latency trading.

References and Further Readings

Memory Allocation Strategies - Part 2

Linear/Arena Allocators

www.gingerbill.org

Arena allocator tips and tricks

nullprogram.com/blog/2023/09/27/ This article was discussed on Hacker News. Over the past year I've refined my approach...

nullprogram.com

Untangling Lifetimes: The Arena Allocator

Making performant dynamic manual memory management in C feel almost like garbage collection.

www.rfleury.com

GitHub - JaMo42/cpp-arena-allocator: Region based C++ allocator

Region based C++ allocator. Contribute to JaMo42/cpp-arena-allocator development by creating an account on GitHub.

github.com

Cpp

Hft

Memory Management





Follow

Written by Ng Song Guan

97 followers · 7 following

Software engineer writing about computer systems. <https://www.linkedin.com/in/song-guan-ng/>
